

Betriebssysteme 2 | BSys2

Zusammenfassung

```
int *p = &x; // &x = Adresse des Ints, * = Pointer-Bezeichner
int y = *p; // *p = Wert einer Int-Adresse, y = 5, * = Dereferenzoperator
```

4096	2048	1024	512	256	128	64	32	16	8	4	2	1
2 ¹²	2 ¹¹	2 ¹⁰	2 ⁹	2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰

1'048'576	65'536	4'096	256	16	1
16 ⁶	16 ⁴	16 ³	16 ²	16 ¹	16 ⁰
10 000 00 ₁₀	01 00 00 ₁₆	00 10 00 ₁₆	00 01 00 ₁₆	00 00 10 ₁₆	00 00 01 ₁₆

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------

1. BETRIEBSSYSTEM API

Aufgaben: Abstraktion, Portabilität, Ressourcenmanagement & Isolation der Anwendungen, Benutzerverwaltung und Sicherheit.
Privilege Levels: *Kernel-Mode* (darf alles ausführen, Ring 0), *User-Mode* (darf nur beschränkte Menge an Instruktionen ausführen, Ring 3)

Kernel: *Microkernel* (nur kritische Teile laufen im Kernel-Mode), *monolithisch* (meiste OS, weniger Wechsel, weniger Schutz), *Hybridkernel* (kann fast nur ein Programm ausführen)

syscall verknüpft den Prozessor, in den Kernel-Mode zu **schalten**. Jedes OS-Kernel-Funktion hat einen Code, der einen Register übergeben werden muss. (*eax* hat den Code 60)

ABI: Application Binary Interface, Abstrakte Schnittstelle mit plattformunabhängigen Aspekten.

API: Application Programming Interface, Konkrete Schnittstellen, Calling Convention, Abbildung von Datenstrukturen. **Linux-Kernels** sind API-, aber nicht ABI-kompatibel. (*C-Wrapper-Funktionen*)

POSIX: Portable Operating System Interface. Sammlung von IEEE Standards, welche die Kompatibilität zwischen OS gewährleisten. Windows ist nicht POSIX-konform.

1.1. PROGRAMMARGUMENTE

`clang -o abc.c -o abc.c` : Die Shell teilt Programmargumente in Strings auf (*Trennung durch Leerzeichen, Quotes*). **Calling Convention:** OS schreibt Argumente als null-terminierte Strings in den Speicherbereich des Programms. Zusätzlich legt das OS ein Array `argv` an, dessen Elemente jeweils auf das erste Zeichen eines Arguments zeigen. Die Art und Weise, wie das gehandhabt wird, ist die **Calling Convention**. Werden explizit angegeben, nützlich für Informationen, die bei jedem Aufruf anders sind.

```
int main(int argc, char **argv) { ... } // argv[0] = program path
```

1.2. UMGEBUNGSVARIABLEN

Strings, die mindestens ein Key=Value enthalten (`OPTERR=1, PATH=/usr/bin`). Der Key muss einzigartig sein. Unter POSIX verwaltet das OS die Umgebungsvariablen innerhalb jedes laufenden Prozesses. Werden initial festgelegt. Das OS legt die Variablen als ein null-terminiertes Array von Pointern auf null-terminierte Strings ab. Unter C zeigt die Variable `extern char **environ` darauf. Sollte nur über unterstehende Funktionen manipuliert werden. Werden implizit bereitgestellt, nützlich für Informationen, die bei jedem Aufruf gleich sind.

– **Abfragen einer Umgebungsvariable:** `char *value = getenv("PATH");`

– **Setzen einer Umgebungsvariable:** `int ret = setenv("HOME", "/usr/home", 1);`

– **Entfernen einer Umgebungsvariable:** `int ret = unsetenv("HOME");`

– **Hinzufügen einer Umgebungsvariable:** `int ret = putenv("HOME=/usr/home");`

(*gefährlich wegen Pointer auf NULL*)

Größere Konfigurationsinformationen sollten über **Dateien** übermittelt werden.

2. DATEISYSTEM API

Anwendungen dürfen nie annehmen, dass Daten gültig sind.

Arbeitsverzeichnis: Bezugspunkt für relative Pfade. Jeder Prozess hat eines (`getcwd()`, `chdir()`: Nimmt String, `chdir()`: Nimmt File Descriptor).

Pfad: Absolut (beginnt mit /), Relativ (beginnt nicht mit /), Kanonisch (Absolut, ohne `..` oder `..`, `realpath()`)

– **NAME_MAX:** Maximale Länge eines Dateinamens (exklusiv terminierender Null)

– **PATH_MAX:** Maximale Länge eines Pfades (inklusive terminierender Null) (*beinhaltet: Wert von NAME_MAX*)

– **POSIX_PATH_MAX:** Minimaler Wert von NAME_MAX nach POSIX (14)

– **POSIX_PATH_MAX:** Minimaler Wert von PATH_MAX nach POSIX (256)

// Gibt Arbeitsverzeichnis aus
int main (int argc, char **argv) { char *wd = malloc(PATH_MAX);
getcwd(wd, PATH_MAX); printf("Current WD is %s", wd); free(wd); return 0;
}

Zugriffsrechte: Je 3 Permission-Bits für Owner, Group und anderen Benutzer. Bits sind: read, write, execute, a=2, w=2, x=1. **Beispiel:** 0740 oder `rwx r-- --` → Owner hat alle Rechte, Group kann lesen, anderen haben keine Rechte. **POSIX:** `S_IRWXU = 0700, S_IRUSR = 0200, S_IRGRP = 0040, S_IXOTH = 0001. Werden mit | verknüpft.`

POSIX-API: für direkten Zugriff, alle Dateien sind rohe Binärdaten. **C-API:** für direkten Zugriff auf Streams. **POSIX File API:** für direkten, unformatierten Zugriff auf Inhalt der Datei, nur für Binärdaten verwendet. **errno:** Makro oder globale Variable vom typ `int`. Sollte direkt nach Aufrufen eines Fehlers aufgerufen werden.

`if (chdir("docs") == 0) { if (errno == EACCESS) { printf("Error: Denied"); } }`

strerror: gibt die Adresse eines Strings zurück, der den Fehlercode `code` textuell beschreibt. **program** schreibt text gefolgt von einem Doppelpunkt und vom Ergebnis von `strerror(errno)` auf den Errorstream.

2.1. FILE-DESCRIPTION (FD)

Files werden in der POSIX-API über FDs repräsentiert. Gibt nur *innerhalb eines Prozesses*. **Returt Index in Tabelle** aller geöffneten Dateien im Prozess → Enthält *Index in systemweite Tabelle* → Enthält *Index zur Identifikation der Datei*. `stdin_FILENO = 0`: Standard input, `stdout_FILENO = 1`: Standard output, `stderr_FILENO = 2`: Standard error

```
int open (char *path, int flags, ...): öffnet eine Datei. Erzeugt FD auf Datei an path.
Flags gibt an, wie die Datei geöffnet werden soll:
+ O_RDONLY: nur Lesen,
+ O_RDWR: Lesen und Schreiben,
+ O_CREAT: Erzeuge Datei, wenn sie nicht existiert,
+ O_APPEND: Setze Offset als Ende der Datei vor jedem Schreibzugriff,
+ O_TRUNC: Setze Länge der Datei auf 0
```

`int close (int fd): schließt Datei bzw. dealloziert den FD. Kann dann wieder für andere Dateien verwendet werden. Wenn FD's nicht geschlossen werden, kann das File-Limit erreicht werden, dann können keine weiteren Dateien mehr geöffnet werden. Wenn mehrere FDs die gleiche Datei öffnen, können sie sich gegenseitig Daten überschreiben.`

```
int fd = open("myfile.dat", O_RDONLY);
if (fd < 0) { /* error handling */ } // read data; // close(fd);
```

size_t read(int fd, void *buffer, size_t n): kopiert die nächsten n Bytes am aktuellen Offset von *fd* in den Buffer.

size_t write(int fd, void *buffer, size_t n): kopiert die nächsten n Bytes vom *buffer* an den aktuellen Offset von *fd*

```
#define N 32
char buf[N];
char spath[PATH_MAX]; // source path
char dpath[PATH_MAX]; // destination path
// ... gets paths from somewhere
int arc = open(spath, O_RDONLY);
int dat = open(dpath, O_WRONLY | O_CREAT, S_IRWXU);
size_t read_bytes = read(arc, buf, N);
write(dat, buf, read_bytes); // if file closed early, only write as many bytes
close(arc); // as have been read from the file
close(dat);
```

off_t lseek(int fd, off_t offset, int origin): *Seiten in einer Datei*. Verschiebt den Offset und gibt den neuen Offset zurück. **SEEK_SET:** Beginn der Datei, **SEEK_CUR:** Aktueller Offset, **SEEK_END:** Ende der Datei. **lseek(fd, 0, SEEK_CUR)** gibt aktuellen Offset zurück, **lseek(fd, 0, SEEK_END)**: gibt die Grösse der Datei zurück.

lseek und read/write: *fd, void *buffer, size_t n, off_t offset*: **Lesen und schreiben ohne Offsetänderung.** Wie read bzw. write. Statt des Offsets von *fd* wird der zusätzliche Parameter *offset* verwendet.

2.1.1. Unterschiedliche Windows und POSIX

Bestandteile von Pfaden werden durch **Backslash (\)** getrennt, ein **Wurzelverzeichnis** *pro* Betriebssystem/Partition, andere File-Handling-Funktionen.

2.2. C-STREAM API

Unabhängig vom Betriebssystem, Stream-basiert, gepuffert oder ungepuffert, hat einen eigenen File-Position-Indicator (FPI).

Streams: FILE + enthält **Informationen über einen Stream**. Soll nicht direkt verwendet oder kopiert werden, sondern nur über C-API erzeugte Pointer.

FILE + fopen(char *const *path, char *mode): Öffnen eine Datei. Erzeugt FILE-Objekt für Datei an *path*. Flags für *mode*: "r" (Datei lesen), "w" (in neue oder bestehende Datei schreiben), "a" ("in neue oder bestehende Datei anfügen", "r+": Datei lesen & schreiben), "a+": (neue oder erweiterte bestehende Datei lesen & überschreiben), "a+": (neue oder bestehende Datei lesen & an Datei anfügen). Gibt Pointer auf erzeugtes FILE-Objekt zurück oder 0 bei Fehler. FILE * `fopen(int fd, char *const * mode)` ist gleiches, aber statt *fd* wird direkt der *FD* übergeben. *int* `fclose (FILE *stream)` gibt *FD* zurück. Nach API-Wechsel die vorherige API nicht mehr verwenden.

int fclose(FILE *stream): Schließt eine Datei. Ruft `fclose()` auf (*schreibt Inhalt aus Speicher in die Datei*), schließt den Stream, entfernt file aus Speicher und gibt 0 zurück wenn OK, andernfalls EOF.

int fgetc(FILE *stream): Liest das nächste Byte und erhöht *FPI* um 1.

char * fgets(char *buf, int n, FILE *stream): liest bis zu n Zeichen aus stream.

int ungetc(int c, FILE *stream): Lesen rückgängig machen. Nicht Unget-Stack.

int fputc(int c, FILE *stream): Schreibt c in eine Datei.

int fputs(char *c, FILE *stream): Schreibt die Zeichen vom String s bis zur terminierenden 0 in stream.

2.2.1. Dateiteile und Streams

int feof(FILE *stream) gibt 0 zurück, wenn *Dateien nicht erreicht* wurde

int ferror(FILE *stream) gibt 0 zurück, wenn *kein Fehler* auftrat.

2.2.2. Manipulation des File-Position-Indicator (FPI):

long ftell(FILE *stream) gibt den gegenwärtigen *FPI* zurück, **int fseek (FILE *stream, long offset, int origin)** setzt den *FPI*, analog zu `lseek`, **int rewind (FILE *stream)** setzt den Stream zurück.

3. PROZESSE

Prozesse (*procs*) sind die **Verwaltungseinheit** des OS für Programme (*process*). Jedem Prozess ist ein **virtueller Adressraum** zugeordnet.

Ein Prozess umfasst das **Abbild eines Programms** im Hauptspeicher (*text section*), die **globalen Variablen des Programms** (*data section*), Speicher für den Heap und Speicher für das **Stack**.

Process Control Block (PCB): Das Betriebssystem hat Daten über jeden Prozess in jeweils einem **PCB** vor. Speicher für alle Daten, die das OS benötigt, um die Ausführung des Prozesses ins Gesamtsystem zu integrieren, u.a.: Diverse IDs, Speicher für Zustand, Scheduling-Informationen, Daten zur Synchronisation, Security-Informationen etc.

Interrupts: *Kontext* des aktuellen Prozesses muss im dazugehörigen PCB gespeichert werden.

Context: Register, Flag, Instruction Pointer, MMU-Kontroller, **Interrupt-Handler** überschreibt den Kontext. Anschliessend wird Kontext aus PCB wiederhergestellt (*context restore*).

Prozess-Erstellung: Das OS erzeugt den Prozess und lädt das Programm in den Prozess.

Unter POSIX getrennt, unter Windows eine einzige Funktion.

Prozess-Hierarchie: Baumstruktur, startet bei Prozess 1.

3.1. PROZESS-API

pid_t fork(void) erzeugt **exakte Kopie (C)** als Kind des Prozesses (*P*), mit eigener Prozess-ID (> 0). Die Funktion führt in **beiden** Prozessen den Code an derselben Stelle fort.

void exit(int code): Beendet das Programm und gibt code zurück.

pid_t wait(int *status): unterbricht Prozess, bis Child beendet wurde.

pid_t waitpid (pid_t pid, int *status, int options): wie `wait()`, aber *pid* bestimmt, auf welchen Child-Prozess man warten will (> 0 = Prozess mit dieser ID, -1 = irgendeinen Child-Prozess, 0 = alle C mit dem gleichen Prozess-ID).

```
void spawn_worker (...) {
    if (fork() == 0) { /* do something in worker process; // exit(0); }
    // ... do something in parent process
    do { pid = wait(0); } while (pid > 0 || errno == ECHILD); //wait for all children
}
```

exec()-Funktionen: Jede davon **ersetzt** im gerade laufenden Prozess das **Programm durch ein anderes Programm**. Informationen müssen spezifiziert werden. (`_Lokale IDs, ... als Array`)

Aufrufen der Executable

Angabe des Pfads mit neuem Environment

Suche über PATH mit altem Environment

3.1.1. Zombie- & Orphan-Prozesse

C ist zwischen seinem Ende und dem Aufruf von `wait()` durch *P* ein **Zombie-Prozess**. **Dauerhafter Zombie-Prozess:** *P* ruft wegen Fehler `wait()` nie auf. **Orphan-Prozess:** *P* wird vor *C* beendet. *P* kann somit nicht mehr auf *C* warten, was bei Beendung von *C* in einem dauerhaften Zombie resultiert. Wenn *P* beendet wird, werden deshalb alle *C* an Prozess mit `pid=1` übertragen, der `wait()` in ein Orphan-schleife aufruft.

unsigned int sleep (unsigned int seconds): unterbricht Ausführung, bis eine Anzahl Sekunden ungefähr verstrichen ist. Gibt auf Schlaf noch vorhandene Sekunden zurück.

int atexit (void *function(void*)): Registriert Funktionen für Aufräumarbeiten vor Ende. **pid_t getpid()** (`getpid()` geben die (Parent)-Prozess-ID zurück.

4. PROGRAMME UND BIBLIOTHEKEN

C-Quelle → Präprozessor → Bereinigte C-Quelle → Compiler → Assembler-Datei → Assembler → Objekt-Datei → Linker → Executable

Präprozessor: Die Ausgabe des Präprozessors ist eine reine C-Datei (Translation-Unit) ohne Makros, Kommentare oder Präprozessor-Direktiven. **Linker:** Der Linker verknüpft Objekt-Dateien (und statische Bibliotheken) zu Executables oder dynamischen Bibliotheken. **Loader:** lädt Executables und eventuelle dynamische Bibliotheken direkt in den Hauptspeicher.

4.1. ELF (EXECUTABLE AND LINKING FORMAT)

Binär-Format, das Kompile spezifiziert. Besteht aus **Linking View** (wichtig für Linker, für Object-Files und Shared Objects) und **Execution View** (wichtig für Loader, für Programme und Shared Objects).

Struktur: Besteht aus **Header, Programm Header Table** (execution view), **Segmente** (execution view), **Section Header Table** (linking view), **Sektionen** (linking view)

4.2. SEGMENTE UND SEKTIONEN

Segmente und Sektionen sind eine andere Einteilung für die gleichen Speicherbereiche. View des **Loaders** sind die **Segmente**, view des **Compilers** die **Sektionen**. Definieren „gleichartige“ Daten. Der **Linker** vermittelt zwischen beiden Views.

Header: Beschreibt den **Aufbau** der Datei. Typ, 32/64-bit, Encoding, Maschinenarchitektur, Entropy, Infos zu den Einträgen in PHT und SHL.

Segmente/Program Header Table und Segmente: Tabelle mit n Einträgen, jeder Eintrag (*n = 2* bytes) beschreibt ein Segment (*Typ*, *Start*, *Ende*, *Offset* und *Größe* in Speicher – kann unterschiedlich zur Dateigrösse sein). Ist Verbindung zwischen Segmenten im RAM und im File. Definiert, wo ein Segment liegt und wohin der Loader es im RAM laden soll.

Segmente werden vom **Loader** dynamisch zur **Laufzeit** verwendet.

Section Header Table und Sektionen: Tabelle mit n Einträgen (*n ≠ n*). Jeder Eintrag (*n = 40 bytes*) beschreibt eine Sektion (*Name, Section-Typ, Flags, Offset und Grösse*...). Werden vom **Linker** verwendet.

String-Tabelle: Bereich in der Datei, der nacheinander **null-terminierte Strings** enthält. Strings werden relativ zum Beginn der Tabelle referenziert.

Symbole & Symboltabelle: Die Symboltabelle enthält jeweils **einen Eintrag je Symbol** (16 Byte = 48 Name, 48 Wert, 48 Grösse, 48 Info).

4.3. BIBLIOTHEKEN

Statische Bibliotheken: Archive von Objekt-Dateien. Name: `lib<name>.a`, referenziert wird nur `<name>`. **Linker** behandelt statische Bibliothiken wie **mehrere Objekt-Dateien**. Ursprünglich gab es **nur statische Bibliotheken** (Einfach zu implementieren, aber Funktionalität lag).

Dynamische Bibliotheken: Linken erst zur Laufzeit bzw. Laufzeit des Programms. Höherer Aufwand, jedoch austauschbar. Executable enthält nur Referenz auf Bibliothek. **Vorteile:** Entkoppelter Lebenszyklus, Schnellere Ladezeiten durch Lazy Loading, Flexibler Funktionsumfang.

4.4. POSIX SHARED OBJECTS API

void * dlopen (char * filename, int mode): öffnet eine dynamische Bibliothek und gibt ein Handle darauf zurück, das auf den folgenden Werten:

– **RTLD_NOW:** Alle Symbole werden beim Laden gebunden

– **RTLD_LAZY:** Symbole werden bei Bedarf gebunden

– **RTLD_GLOBAL:** Symbole können beim Binden anderer Objektdateien verwendet werden

– **RTLD_LOCAL:** Symbole können nicht für andere Objektdateien verwendet

void * dlsym (void * handle, char * name): gibt die **Adresse des Symbols** name aus der mit handle bezeichneten **Bibliothek** zurück. Neue Typinformationen (*Variable? Funktion?*)

// type "func_t" is an address of a function with a int param and int return type
typedef int (*func_t)(int);
handle = dlopen("libmylib.so", RTLD_NOW); // open library
func_t f = dlsym(handle, "my_function"); // write my_function addr. into a func_t
int *i = dlsym(handle, "my_int"); // get address of "my_int"
(*f)(4); // call "my_function" with "my_int" as parameter

int dclose (void * handle): schliesst das durch handle bezeichnete, zuvor geöffnete Objekt. **char * dlerror():** gibt Fehlermeldung als null-terminierten String zurück.

Konventionen: Shared Objects können **automatisch** bei Bedarf geladen werden. Der Linker verwendet den Linker-Namen, der vom **Loader** verwendet den SO-Namen.

– **Linker-Name:** lib + Bibliothekname + .so (z.B. libmylib.so)

– **SO-Name:** Linker-Name + . + Versionsnummer (z.B. libmylib.so.2)

– **Real-Name:** SO-name + . + Versionsnummer (z.B. libmylib.so.2.1)

Shared Objects: Heute alle Executables benötigen **zwei Shared Objects:** `libc.so`: Standard C library, `ld-linux.so`: ELF Shared Object loader (*Lad Shared Objects und rekursiv alle Dependencies*).

Implementierung dynamischer Bibliotheken: Müssen verschierbar sein, mehrere müssen in den gleichen Prozess geladen werden. Die Ausgabe des Linkers wird in den Loader bzw. Dynamic Linker verschoben (*Load Time Relocation*).

4.5. SHARED MEMORY

Dynamische Bibliotheken sollen **Code zwischen Programmen teilen**. Code soll **nicht mehrfach** im Speicher abgelegt werden. Mit Shared Memory kann ein Programm eine **eigene virtuelle Page** für die Code definieren. Diese werden auf denselben Frame im RAM gemappt. **Benötigt Position-Independent Code** (Adressen nur relativ zum Instruction Pointer, Prozessor muss relative Instruktionen anbieten).

Relative Moves via **Relative Calls:** Mittels Hilfsfunktion wird Rücksprungadressen in Register abgelegt, somit kann relativ dazu gearbeitet werden.

Global Offset Table (GOT): Pro dynamische Bibliothek & Executable vorhanden, enthält pro Symbol einen Eintrag. Der Loader führt zur Laufzeit die Adressen in die GOT ein.

Procedure Linking Table (PLT): Implementiert **Lazy Binding**. Entspricht pro Funktion einen Eintrag, dieser enthält Sprungbefehl an Adresse in GOT-Eintrag. Dieser zeigt auf eine Proxy-Funktion, welche den GOT-Eintrag überschreibt. **Vorteil:** expand bedingten Sprung.

5. THREADS

Jeder Prozess hat virtuell den **ganzen Rechner für sich alleine**. Prozesse sind gut geeignet für **unabhängige Applikationen**. Nachteile: Realisierung **paralleler Abläufe** innerhalb derselben Applikation ist **aufwändig**. **Overhead** zu gross falls nur kürzere Taskläufe, **gemeinsame Ressourcen** nicht erschwert.

Threads: parallel ablaufende Aktivitäten innerhalb eines Prozesses, welche auf alle Ressourcen im Prozess gleichermassen Zugriff haben. Benötigen **eigenen Kontext und eigenen Stack**. Informationen werden in einem **Thread-Control-Block** abgelegt.

5.1. AMDAHL'S REGEL

Nur bestimmte Teile eines Algorithmus können parallelisiert werden.

T₁ : Ausführungszeit, wenn **komplett seriell** durchgeführt (im Bild *T₁ = T₁ + T₁ + T₁ + T₁*)

n : Anzahl der Prozessoren

T_p : Ausführungszeit, wenn **maximal parallelisiert** gedachte Grösse

T_p : Ausführungszeit für den Anteil, der **seriell** ausgeführt werden muss (im Bild *T_p = T₁ + T₂ + T₃*)

T₁ - T_p : Ausführungszeit für den Anteil, der **parallel** ausgeführt werden kann (im Bild *T₁ - T₂*)

$$(T - T_p) \cdot n = \text{Parallel-Anteil verteilt auf alle n Pro- zessen. (im Bild: } (T_1 - T_2) \cdot n)$$

$$T_p + \frac{T - T_p}{n} = \text{Serieller Teil + Paralleler Teil} = T$$

Die **serielle Variante** benötigt also höchstens **f-mal** mehr Zeit als die **parallele Variante**:

$$f \leq \frac{T}{T_p} = \frac{T}{T_p + \frac{T - T_p}{n}} = \frac{T}{T_p} \cdot \frac{n}{n} = \frac{T}{T_p} \cdot \frac{n}{n} = \frac{T}{T_p} \cdot \frac{n}{n} \Rightarrow f \leq \frac{1}{\frac{T_p}{T} + \frac{1}{n}}$$

f heisst auch **Speedup-Faktor**, weil die parallele Variante max. *f*-mal schneller ist als die serielle.

Definiert man $n = T_1/T_p$, also den seriellen Anteil am Algorithmus, dann ist $n \cdot T = T_1$. Dadurch erhält man 1 unabhängig von der Zeit:

$$f \leq \frac{1}{\frac{T_p}{T} + \frac{1}{n}} = \frac{T}{T_p + \frac{T - T_p}{n}} = \frac{T}{T_p} \cdot \frac{n}{n} = \frac{T}{T_p} \cdot \frac{n}{n} = \frac{T}{T_p} \cdot \frac{n}{n} \Rightarrow f \leq \frac{1}{\frac{T_p}{T} + \frac{1}{n}}$$

Parallelität (n = 4)

Die serielle Variante benötigt also höchstens f-mal mehr Zeit als die parallele Variante:

$$f \leq \frac{T}{T_p} = \frac{T}{T_p + \frac{T - T_p}{n}} = \frac{T}{T_p} \cdot \frac{n}{n} = \frac{T}{T_p} \cdot \frac{n}{n} = \frac{T}{T_p} \cdot \frac{n}{n} \Rightarrow f \leq \frac{1}{\frac{T_p}{T} +$$

